COMPUTER VISION • NOVA School of Science and Technology • 2026/2027

# Project Stage 1: Cut!

Detecting shot boundaries and recurring scenes in edited video


**Deadline** November, 2nd, 2026

**Contacts** David Semedo df.semedo@fct.unl.pt • Rui Nóbrega rui.nobrega@fct.unl.pt



**Version** v1 (October 14, 2026)


## 1 Introduction

An edited video is a sequence of *shots*: continuous takes from a single camera, glued together by *transitions*. The most common transition is the hard *cut*, where one frame belongs to a shot and the next frame to another; in this project, all transitions are cuts. Finding these boundaries, a task called *shot boundary detection*, is the first step of almost every video-analysis system: indexing, summarisation, thumbnail selection, or ad detection.

In this project you will build a cut detector using only the tools from the first labs: image processing, colour histograms, convolution and edge detection (Canny), and local features (Harris, SIFT). You will then go one step further and find shots that show the *same scene*, because editors often return to a location several times. This second problem is the main focus of the project. Above all, you will **measure** how well each idea works, using precise metrics and ground truth, and report your findings in a notebook.

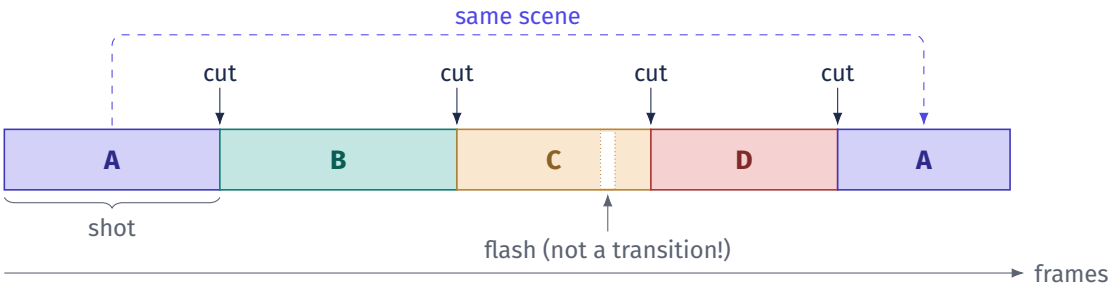



Figure 1: The anatomy of an edited video. Each coloured block is a shot; letters denote the scene (source take). The task is to find every cut, ignore distractors such as flashes, and group the shots that show the same scene.

## 2 Rules

- The project is done in **groups of 2** (except student-workers, as registered in CLIP). Single-student groups are only accepted with explicit authorisation from the professors by email.
- Only the libraries used in the labs are allowed: **OpenCV, NumPy and Matplotlib**, plus standard Python libraries (`json`, `os`, `glob`, ...). Functions you are asked to *implement* (e.g., `histogram_distance`) must be written by you, not called from OpenCV.

- Any use of AI tools beyond looking up what a function does or correcting language must be **declared** in a dedicated section of the report, stating what was used and for what.
- You must follow the code of ethics of the Department of Computer Science.

The code must follow this structure (the starter code on Moodle already does):

```
.
|-- input/                  # read-only: never write here
|   |-- dev_01.mp4   dev_01.json
|   |-- dev_02.mp4   dev_02.json
|   |-- test_01.mp4  test_01.json
|   `-- test_02.mp4  test_02.json
|-- output/                 # everything your code writes goes here
`-- 50000_60000/            # your student numbers, smaller first: this is what you submit
    |-- tp1.py              # entry point: python tp1.py
    |-- metrics.py          # evaluation code (Section 7), given
    |-- plotting.py         # plotting helpers (given)
    |-- grouping.py         # union-find grouping of shots (given)
    |-- ...                 # any other .py files you need
    `-- report.ipynb
```

## 3 Dataset and ground truth

The dataset contains four edited videos ($640 \times 360$ pixels, 25 frames per second, about 2.5 minutes each, ≈3 700 frames), assembled from openly licensed footage from Wikimedia Commons (authors and licences are listed in `input/CREDITS.md`).

| Video | Use it for | Contents |
|---|---|---|
| `dev_01`, `dev_02` | designing and tuning your methods | 70 cuts; 72 shots from 47 scenes (19 scenes appear more than once); 12 flashes |
| `test_01`, `test_02` | final numbers only | 78 cuts; 80 shots from 60 scenes (17 scenes appear more than once); 12 flashes |

The test videos use footage that never appears in the development videos. They also contain hard cases on purpose: recurring scenes filmed from a different point of the same take, cuts between different takes of the same location (similar colours), fast camera motion, flashing lights, and camera flashes added by us.

> ⚠ **Important** Tune thresholds and make design decisions **only** on `dev_*`. Report test results once, at the end. Choosing parameters by looking at test scores is a methodological error and will be penalised.

### 3.1 Ground-truth format

Each video comes with a JSON file. Frames are **0-indexed** (see Task 1).

```
{
  "video": "dev_01.mp4", "fps": 25, "num_frames": 3809,
  "transitions": [
    {"type": "cut", "start": 74,  "end": 74},   // 1st frame of new shot
    {"type": "cut", "start": 188, "end": 188},
```

```
    ...
  ],
  "shots": [
    {"id": 0, "start": 0,   "end": 73,  "scene": "S00"},
    {"id": 1, "start": 74, "end": 187, "scene": "S01"},
    ...
  ],
  "distractors": [ {"type": "flash", "start": 860, "end": 861}, ... ]
}
```

- All transitions are **cuts**. A cut is stored with `start = end =` the first frame of the new shot.
- Two shots have the same `scene` label if, and only if, they were cut from the same source take.
- `distractors` are **not** transitions: they mark events inside a shot that tend to fool detectors.

## 4 Part 1: Frame dissimilarity and hard cuts

A cut is a sudden change between two consecutive frames. The question is *how to measure change*. You will implement three ways of measuring it and compare them.

### Task 1 Reading and preparing frames

Read the videos frame by frame with `cv2.VideoCapture` (the starter code does this in `read_frames`). Working at full resolution is not necessary: downscale the frames (e.g., to $320 \times 180$) and justify the choice. The whole program must run within the time limit (Section 8).

**Frame numbers are 0-indexed**: the first frame of the video is frame $0$, the second is frame $1$, and the last is frame `num_frames-1`. Frame $t$ is simply `frames[t]`, where `frames` is the list returned by `read_frames`. Every frame number in the ground truth and in your predictions follows this convention. For example, the cut `{"start": 74}` means that `frames[73]` is the last frame of the old shot and `frames[74]` is the first frame of the new one; a signal value $d(74)$, which compares frames 73 and 74, should therefore be high.

### Task 2 Three dissimilarity signals

A *dissimilarity signal* gives one number per frame, measuring how different that frame is from the previous one:

$$d(t) = \text{dissimilarity}\big(\texttt{frames}[t-1],\ \texttt{frames}[t]\big), \qquad t = 1, \ldots, \texttt{num_frames} - 1,$$

with $d(0) = 0$, since frame $0$ has no previous frame. $d(t)$ is small inside a shot and large when frame $t$ is the first frame of a new shot. Implement three dissimilarity functions, and compute one signal $d(t)$ with each of them:

- `pixel_difference(f1, f2)`: mean absolute difference of grey levels. This is your baseline.
- `histogram_distance(h1, h2)`: compares two colour histograms using the $\chi^2$ distance. **Implement it yourself** with NumPy. Choose the colour space (RGB, HSV, …) and the number of bins, and justify the choice.
- `edge_change_ratio(f1, f2)`: the Edge Change Ratio (ECR), described below.

**The Edge Change Ratio.** The idea is simple. Inside a shot, objects and the camera move only a few pixels between two consecutive frames, so almost every edge of frame $t$ lies *close to* an edge of frame $t-1$. After a cut, the edges of the new frame are in unrelated places. The ECR

measures the fraction of edges that appear "out of nowhere" (entering) or disappear (exiting). It is computed in four steps:

1. Convert both frames to grey levels and compute their Canny edge maps $E_{t-1}$ and $E_t$ (binary images). $|E|$ denotes the number of edge pixels in $E$.
2. Dilate each edge map (`cv2.dilate`, e.g., with a $5 \times 5$ kernel) to obtain $\hat{E}_{t-1}$ and $\hat{E}_t$. Dilation thickens every edge, so an edge that moved by one or two pixels still falls inside the thickened version of the old one.
3. Count the *entering* edge pixels, edges of frame $t$ that are not close to any edge of frame $t-1$, and the *exiting* edge pixels, edges of frame $t-1$ that are not close to any edge of frame $t$:
$$X_t^{in} = |E_t \setminus \hat{E}_{t-1}|, \qquad X_t^{out} = |E_{t-1} \setminus \hat{E}_t|,$$
where $A \setminus B$ is the set of pixels that are in $A$ and not in $B$. With Boolean NumPy arrays, $X_t^{in}$ is `np.sum(E_t & ~D_prev)`, with `D_prev` the dilated $E_{t-1}$.
4. Normalise each count by the number of edges of its own frame and keep the largest:
$$\mathrm{ECR}_t = \max\left(\frac{X_t^{in}}{|E_t|}, \frac{X_t^{out}}{|E_{t-1}|}\right) \in [0, 1].$$
Decide what to return when a frame has no edges at all (e.g., a black frame), so that you never divide by zero.

Figure 2 shows two examples from `dev_01`. Inside a shot (top), only $279$ of the $5\,393$ edges of frame $t$ are entering and $204$ of the $5\,279$ edges of frame $t-1$ are exiting. Across a cut (bottom), most edges are new:

$$\begin{aligned} \text{inside a shot:} \quad & \mathrm{ECR} = \max\left(\tfrac{279}{5393}, \tfrac{204}{5279}\right) = \max(0.05,\, 0.04) = 0.05, \\ \text{across a cut:} \quad & \mathrm{ECR} = \max\left(\tfrac{4261}{7973}, \tfrac{1658}{5258}\right) = \max(0.53,\, 0.32) = 0.53. \end{aligned}$$

Both shots in the bottom row are mostly green, so their colour histograms are similar, but their edges are not. This is why it pays to compare several signals.




Figure 2: The Edge Change Ratio on two pairs of consecutive frames. Right column: dilated edges of frame $t-1$ in grey; edges of frame $t$ in green when they fall inside them (the edge moved little) and in red when they do not (entering edges, $X_t^{in}$).

### Task 3 From signal to cuts

Detect cuts by thresholding each signal: a cut is declared at frame $t$ when $d(t) > T$, where $T$ is a **threshold that you must choose**, one per signal (the three signals have very different ranges). Choose it on the development videos: try many values of $T$, compute the precision, recall and $F_1$ of the resulting cuts for each value, and keep the value with the best $F_1$. A low $T$ finds every cut but also many false alarms (high recall, low precision); a high $T$ does the opposite. The same sweep gives the precision–recall curve of the deliverable below. Combining signals is allowed and encouraged (e.g., a cut when two signals are both above their thresholds); then choose the thresholds together, in the same way.

**Deliverable** A plot of the three signals over about 30 seconds of a dev video, with the ground-truth cuts and flashes overlaid (use the function `plot_signals` given in `plotting.py`); a table with the precision, recall and $F_1$ of cut detection for each signal; and a **precision–recall curve** obtained by sweeping the threshold of your best signal. Compute the scores with the functions given in `metrics.py` (Section 7): `match_cuts` gives TP, FP and FN for a list of predicted cut frames, and `precision_recall_f1` turns them into scores. To score both dev videos at once, evaluate each one with `evaluate_video`, add the counts with `sum_counts`, and print the results with `print_table` if you like. Each threshold value of the sweep gives one point of the curve. Draw the curves with `plot_pr_curve`, given in `plotting.py`: you pass it the (threshold, precision, recall) points of each signal and it marks the threshold with the best $F_1$. Its docstring shows how to compute the points and how to call it.

## 5 Part 2: False alarms

Flashes, fast camera motion and objects crossing the frame create large differences *inside* a shot, and a threshold low enough to catch every cut will also fire on them.

### Task 4 Verification with local features

Many false alarms happen *within a single shot*: the camera moves, a flash goes off, a bus crosses the frame. In these cases the content before and after the candidate is still the same scene. Your detector from Part 1 gives *candidate cuts*: frames $t$ where it believes a new shot starts. Verify each candidate as follows:

1. Take one frame **before the candidate**, $t - g$, and one frame **after it**, $t + g$, with a small gap $g$ (e.g., $g = 4$). If $t$ is a real cut, frame $t - g$ belongs to the old shot and frame $t + g$ to the new one. If it is a false alarm, both frames belong to the same shot. Do not use frames $t - 1$ and $t$ themselves: a flash lasts up to 3 frames, and the gap makes sure that neither of the two frames is a flash frame.
2. Match SIFT (or Harris-based) features between frames $t - g$ and $t + g$, keeping the matches that pass Lowe's ratio test. Optionally, count only the RANSAC inliers of a homography (`cv2.findHomography`).
3. If the two frames match well (many matches: same content), **reject** the candidate; otherwise keep it as a cut. Choose the minimum number of matches on the development videos.

**Deliverable** A table with precision, recall and $F_1$ of cut detection with and without verification (`match_cuts` and `precision_recall_f1`, or `evaluate_video` and `sum_counts` for the dev totals); the number of flash distractors that triggered a false detection, with and without verification (`evaluate_video` already computes it, by calling `flash_false_alarms`, and returns

it as `flash_fa`; it is the `flash-FA` column of `print_table`); and one example of a false alarm removed by verification (show the frames and the matches, e.g., with `cv2.drawMatches`).

## 6 Part 3: Recurring scenes

Editors frequently cut back to a scene shown earlier (Figure 1, scene A). In this part you group the shots that come from the same scene. To separate this part from the previous ones, **use the ground-truth shots** (`shots` in the JSON file). This is the core of the project: a recurring shot may show the scene from a different moment of the same take, with the camera moved, people walking by or the light changed, while shots of *different* scenes may look alike (two beaches, two streets in the rain).

### Task 5 Shot representation

Represent each shot by keyframes. Compare at least two strategies: a single keyframe (e.g., the middle frame) and several keyframes sampled from the shot. With several keyframes, you must decide how the keyframe similarities (Task 6) are combined into a single similarity between two shots. **Be creative**: the best pair of keyframes, the average over all pairs, the number of keyframe pairs that match, a vote, … are all valid options. Justify your choice with results on the development set.

### Task 6 Shot similarity

Similarities are always computed **between keyframes**, never between whole shots. For two keyframes, compute: (a) a colour similarity from your histogram distance (e.g., $1 - \chi^2$, since a small distance means very similar); and (b) a local-feature similarity: the number of SIFT matches between the two keyframes that survive Lowe's ratio test. Then combine the keyframe similarities of two shots as decided in Task 5, which gives one number per pair of shots, i.e., a shot×shot similarity matrix.

### Task 7 Grouping

Link every pair of shots whose similarity passes a threshold, and form groups with connected components: if shot 1 is linked to shot 5 and shot 5 to shot 9, the three shots form one group. **Each group is one predicted scene**, and a shot linked to no other shot forms a group on its own. This is done by the function `group_shots(similarity, threshold)`, given in `grouping.py` (union–find). It returns one *scene label* per shot ($0, 1, 2, \ldots$): shots with the same label are in the same group. The labels are just names: they do not have to match the ground-truth names (`S00`, `S01`, …), because the metric (Section 7) only checks whether two shots have the *same* label. Write them in the `scenes` field of your output (Section 8):

```
labels = grouping.group_shots(sim, threshold)       # sim: your shot x shot similarity matrix
scenes = {str(s["id"]): label for s, label in zip(gt["shots"], labels)}
```

Your work in this task is choosing the threshold (on the development set) and analysing the groups you obtain.

**Deliverable** A table with the pairwise precision, recall and $F_1$ (`pairwise_scene_counts` and `precision_recall_f1`, Section 7; `evaluate_video` returns the same counts as `scene`, and `sum_counts` adds them over the dev videos) of the four combinations: histograms and local features, each with one and with several keyframes per shot; the similarity matrix of your best method next to the ground-truth matrix (1 where two shots are in the same scene);

and a “storyboard” figure showing one keyframe per shot coloured by predicted scene, with at least one success and one failure discussed.

## 7 Evaluation metrics

All groups must compute the metrics in exactly the same way, so that results are comparable. The evaluation code is therefore **given** in `metrics.py`, and `tp1.py` already calls it. Do not change it: your job is to produce the predictions. `metrics.py` contains:

- `match_cuts`, `flash_false_alarms`, `pairwise_scene_counts` and `precision_recall_f1`, which compute the counts and the scores (described below);
- `evaluate_video`, which applies them to one video; `sum_counts`, which sums the counts of several videos; and `print_table`, which prints the summary table of Section 8.

You can eventually use these functions in your notebook, for example to evaluate each signal and each threshold in Part 1. Read them: understanding exactly what is measured is part of the project.

Both metrics compare your output with the ground truth and count *true positives* (TP, correct detections), *false positives* (FP, wrong detections) and *false negatives* (FN, missed detections). From these counts:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}.$$

Precision $P$ is the fraction of your detections that are correct; recall $R$ is the fraction of the ground truth that you found; $F_1$ is their harmonic mean, high only when both are high.

**Cut detection.** A predicted cut at frame $p$ and a ground-truth cut at frame $g$ *match* if $|p - g| \leq \tau$, with a tolerance of $\tau = 2$ frames. Each ground-truth cut can be matched by at most one prediction and vice versa (otherwise, predicting the same cut five times would count five times). Conflicts are resolved greedily, pairing the closest ones first. Matched predictions are TP, unmatched predictions are FP, and unmatched ground-truth cuts are FN. This is what the given function `match_cuts` computes.

*Example.* Ground truth $\{74, 188, 355\}$, predictions $\{75, 190, 300, 356, 357\}$. The pairs within $\tau$ are $74$–$75$, $188$–$190$, $355$–$356$ and $355$–$357$. Greedy matching keeps $74$–$75$, $355$–$356$ and $188$–$190$; $357$ (a duplicate of $355$) and $300$ are left alone. So $TP = 3$, $FP = 2$, $FN = 0$, which gives $P = 0.6$, $R = 1$ and $F_1 = 0.75$.

**Flash false alarms.** A flash distractor counts as a false alarm if at least one predicted cut falls within $\tau$ frames of it, i.e., in $[\texttt{start} - \tau,\ \texttt{end} + \tau]$. Report the number of flashes that triggered a false alarm (e.g., 1/6). This is computed by the given function `flash_false_alarms`.

**Scene grouping.** Consider every unordered pair of shots $\{i, j\}$. The pair is a TP if both the ground truth and your prediction put $i$ and $j$ in the same scene, a FP if only your prediction does, and a FN if only the ground truth does. Pairs that are in different scenes in both are not counted. These are *pairwise* precision and recall, computed by the given function `pairwise_scene_counts`.

*Example.* Four shots with ground truth $(A, B, A, C)$ and prediction $(x, y, x, x)$. Of the six pairs, $\{0, 2\}$ is a TP (same scene in both); $\{0, 3\}$ and $\{2, 3\}$ are FP (grouped by the prediction only); there is no FN. So $P = 1/3$, $R = 1$ and $F_1 = 0.5$.

**Totals.** Totals over several videos (e.g., the two test videos) are computed by *summing* TP, FP and FN over the videos and then applying the formulas, *not* by averaging the per-video scores. This is what `sum_counts` does.

**Using the metrics.** Typical use, as in `tp1.py`:

```
counts = metrics.evaluate_video(gt, pred)   # {"cut": (TP, FP, FN), "flash_fa": .., "num_flashes":
    ..,
                                            #  "scene": (TP, FP, FN)}
p, r, f1 = metrics.precision_recall_f1(*counts["cut"])
dev_total = metrics.sum_counts([counts_dev_01, counts_dev_02])
metrics.print_table([("dev_01", counts_dev_01), ("DEV", dev_total)])
```

**Tip** A quick way to understand the metrics: feed the ground truth as a prediction ($P = R = F_1 = 1$); shift every cut by 3 frames ($F_1 = 0$ for cuts); predict a single scene for all shots ($R = 1$, low $P$).

## 8 The program

Running `python tp1.py` from inside your folder must, for every video in `../input`:

1. write `../output/<video>.json` with your predictions, in the same format as the ground truth:

```
{ "video": "test_01.mp4",
  "transitions": [ {"type": "cut", "start": 75, "end": 75},
                   {"type": "cut", "start": 188, "end": 188}, ... ],
  "scenes": { "0": "a", "1": "b", "2": "a", ... } }  // shot id -> label
```

2. evaluate each prediction and print a summary table: one line per video, then the totals of the development and test sets. This part is already written in `tp1.py` (with `evaluate_video`, `sum_counts` and `print_table`); you only need to fill in `process_video`:

```
video      cut-P  cut-R  cut-F1  flash-FA  scene-P  scene-R  scene-F1
dev_01     0.912  0.884   0.898       1/6    0.950    0.700     0.806
...
DEV        0.905  0.871   0.888      2/12    0.941    0.712     0.811
TEST       ...
```

Use the best configuration you found on the development set. The whole run (four videos) must finish in **less than 5 minutes** on a standard laptop. The input folder is read-only, and paths must be relative. Always sort file lists (`sorted(glob.glob(...))`) so that the behaviour is the same on every OS.

## 9 Report

The report is a Jupyter notebook (`report.ipynb`) that imports your code and *shows* your findings with live code, figures and tables. It is not a printout of your code. It must contain:

1. **Identification** of the group and an **introduction**.
2. For each part: a short explanation of the method, illustrated on concrete frames from the videos, and the deliverables listed in the coloured boxes above.
3. **Results**: one final table with the development and test scores of your best configuration.

4. **Analysis**: what works, what does not, and why. Discuss at least three failure cases with frames. Which cuts and which kinds of scenes are hard? Which signal is the most robust to motion and flashes?
5. **Conclusion** with a short self-evaluation, an **extras** section (if any) and the **AI declaration**.

Plots must have axis labels and legends. Every number you report must be produced by your code.

> ⭐ **Extras (bonus)** **Extras are optional and scored as a bonus**. Describe them in the *Extras* section of the report, with results. Some ideas: an adaptive threshold for cuts, computed from a sliding window around each frame; choosing keyframes adaptively (e.g., a new keyframe whenever the content changes); combining colour and local features into a single shot similarity; a sensitivity analysis of the tolerance $\tau$; running your detector on a real film trailer and discussing the results qualitatively.

## 10 Submission and evaluation

- Submit a `zip` of your numbered folder only (e.g., `50000_60000.zip`, without `input`/`output`) through the **Moodle submission form**, which will be made available on Moodle, by **November 2nd, 2026**.
- Your code will be run with an empty `output` folder: `python tp1.py`, then the notebook is executed. All delivered elements are evaluated, including code quality: clean, organised, and commented where needed.
- The grade is 0–20. Completing everything that is asked gives up to 18. Grades of 19 and 20 are reserved for outstanding work that goes clearly beyond the requirements.

**Suggested plan.** The project takes two weeks. Part 3 is the core of the project, so start it early in the second week.

- **Week 1:** read the starter code and `metrics.py`; Part 1 (signals, thresholds, PR curves) in the first days, then Part 2 (SIFT verification). By the end of the week, `tp1.py` should already write the cuts.
- **Week 2:** Part 3 (keyframes, similarities, grouping) in the first half of the week; then the final results on the test videos, the analysis of failure cases and the report.

Write the report as you go: keep the plots and tables of each part in the notebook as soon as they are ready.

**Changelog** v1 (October 14, 2026): first version.